# CS & IT ENGINEERING

## Programming in C

**Arrays and Pointers**
**Lec- 03**

By- Pankaj Sharma sir


Arrays-III

# Pointers


```
int *a;
```

[ Special variables used to store address of other variable. ]



P is a pointer to integer

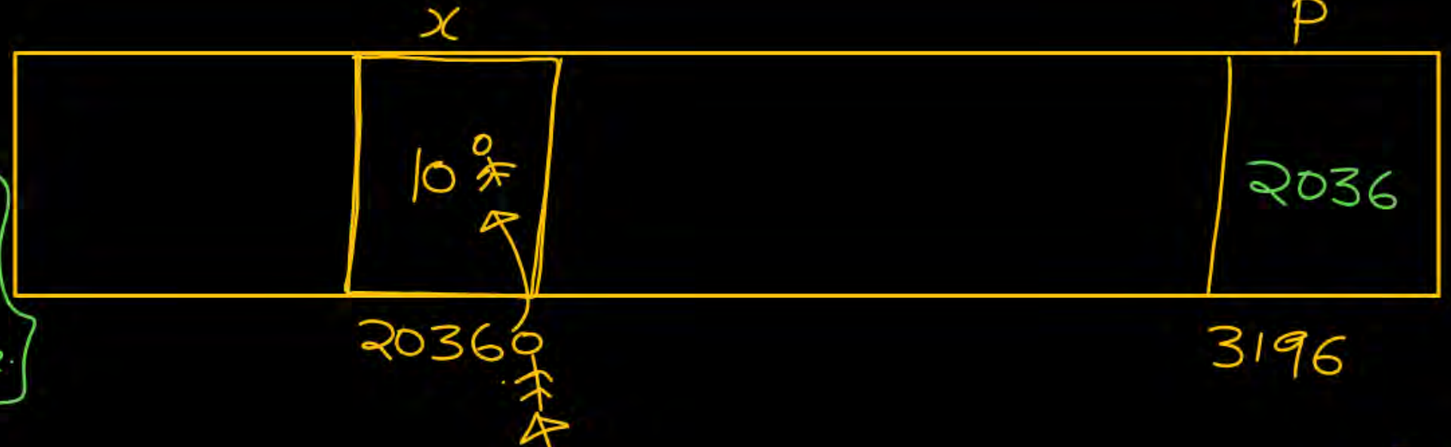
[ P is a pointer variable that can store address of integer variable. ]

```
float p ;
```

```
float *q ;
```

- q is a pointer to float.
- q can store address of float Variable.

```
int x = 10;
int *p;
```
p can store address of integer variable



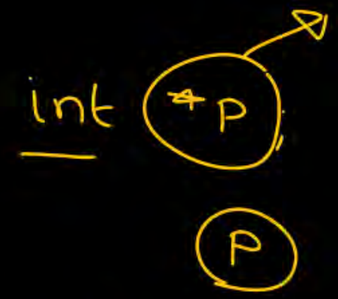

p = &x ;

```
printf("%d", x);   10
printf("%u", p);   2036
printf("%u", &x);  2036
printf("%u", *p);  10
```

p = Memory location 2036

*p = value at (Memory location 2036)

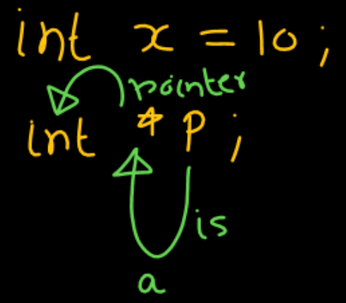
*p ⇒ 10




p = &x ;

*p = *&x ;

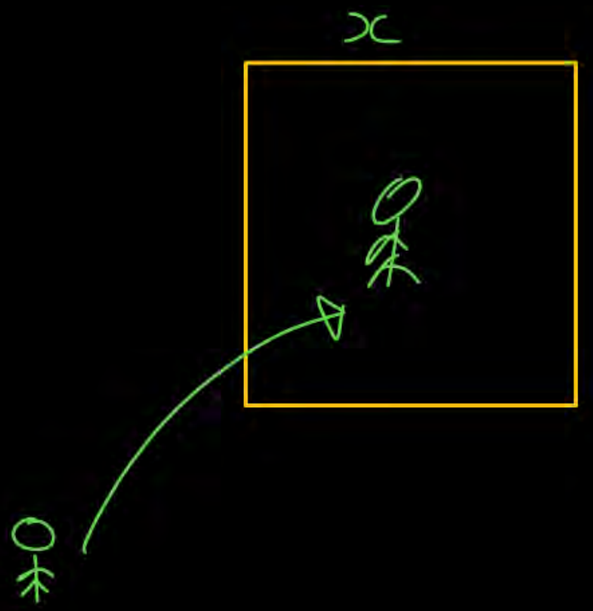
*p ⇒ x



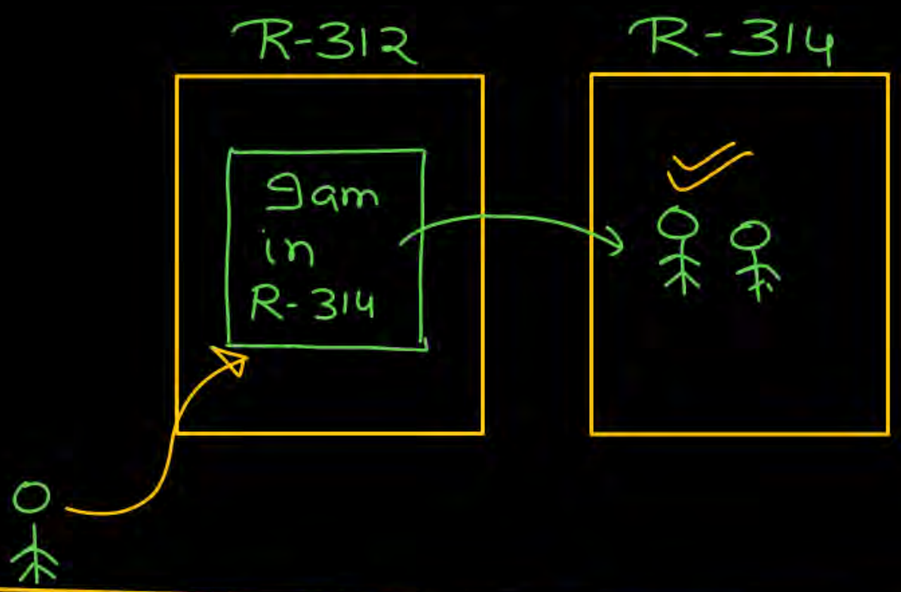


q is a pointer to pointer to int.

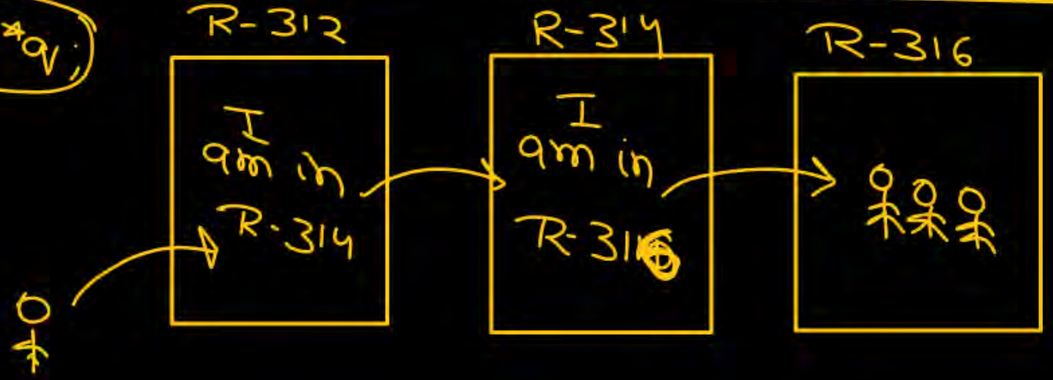
int x = 10;



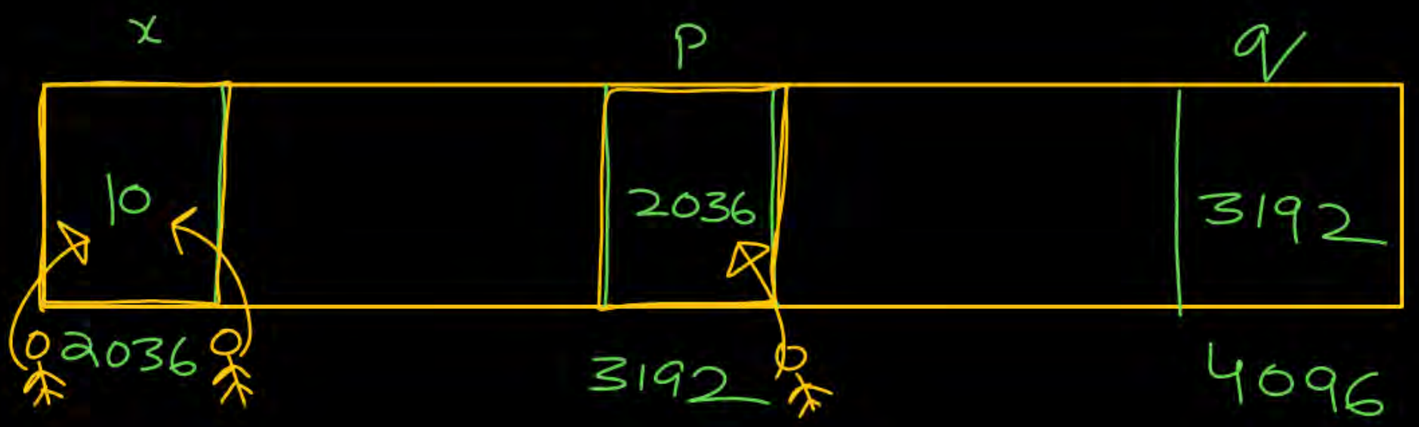

int *p;




int **q;



```
int x = 10;
int *p;
int **q;
p = &x;
q = &p;
```




```
printf("%u", &x);   2036
printf("%u", p);    2036
printf("%u", q);    3192
printf("%u", *p);   10
printf("%u", **q);
```

p = Memory location (2036)

*p = value at (Memory location 2036) = 10

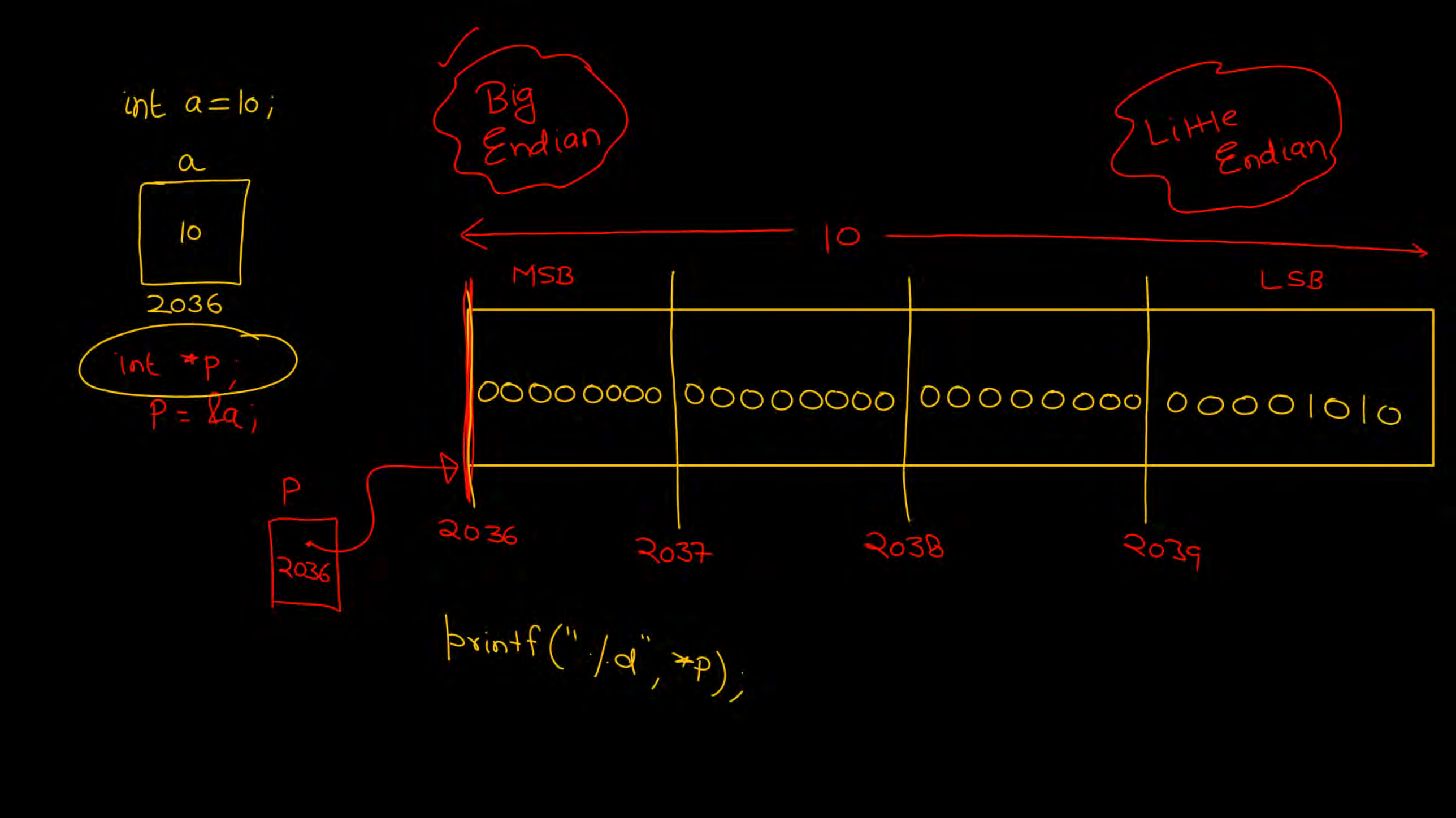
q = {Memory location 3192}

*q = value at (Memory location 3192)

*q = (Memory location 2036)

**q = value at (Memory location 2036)

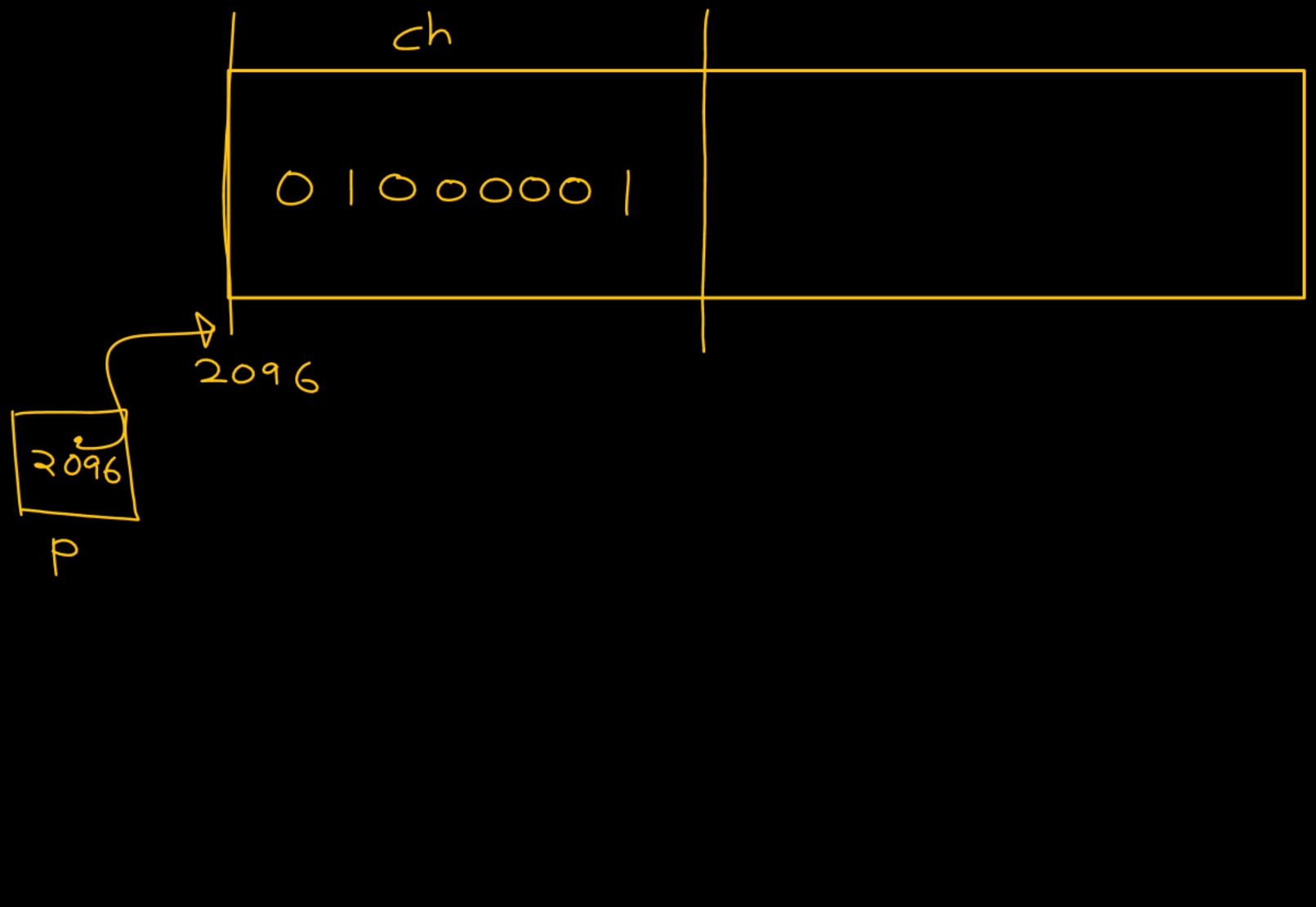
**q = 10


int a=10;
a
10
2036
int *p;
p = &a;
Big Endian
Little Endian
10
MSB
LSB
00000000
00000000
00000000
00001010
P
2036
2036
2037
2038
2039
printf("%d", *p);

char ch = 65;

char *p;

P = &ch;



256 + 32 + 8 + 4

```
int x = 300;
char *p;
p = (char*)&x;
```

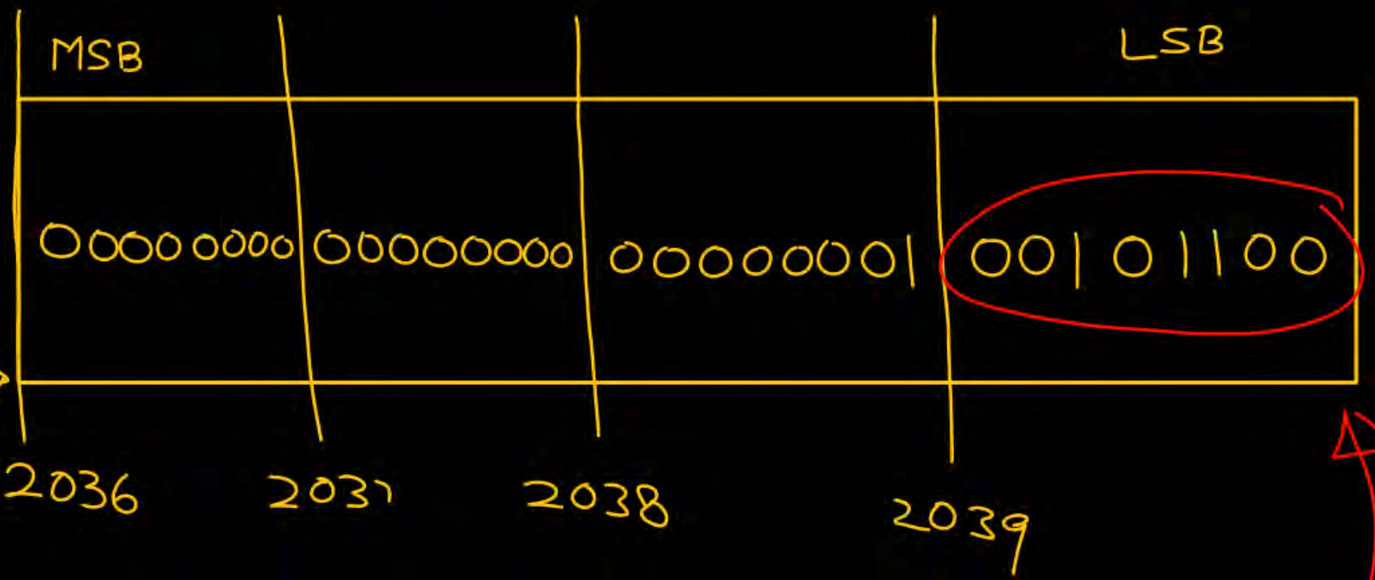
Compiler will shout

To calm down the compiler

Big Endian

```
printf("%d", *p);
```

1 byte




p → 2036

Big Endian 1) 0

Little Endian 2) 44

Little Endian

p

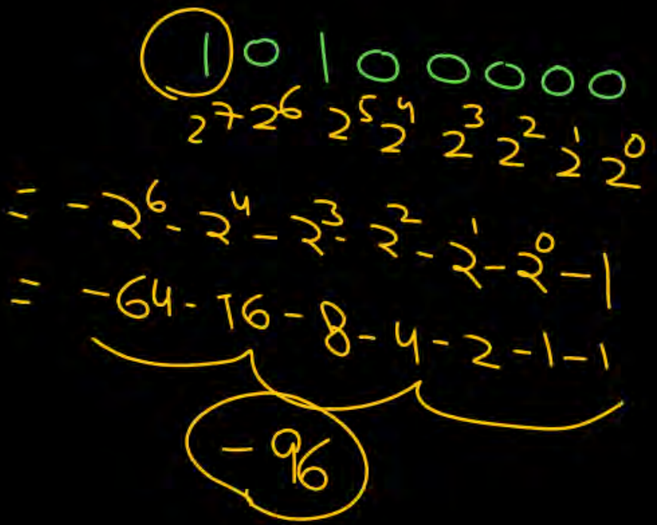
(2)

```
int x = 160;
char *p;
p = (char*)&x;
printf("%d", *p);
```

160 → 128 + 32

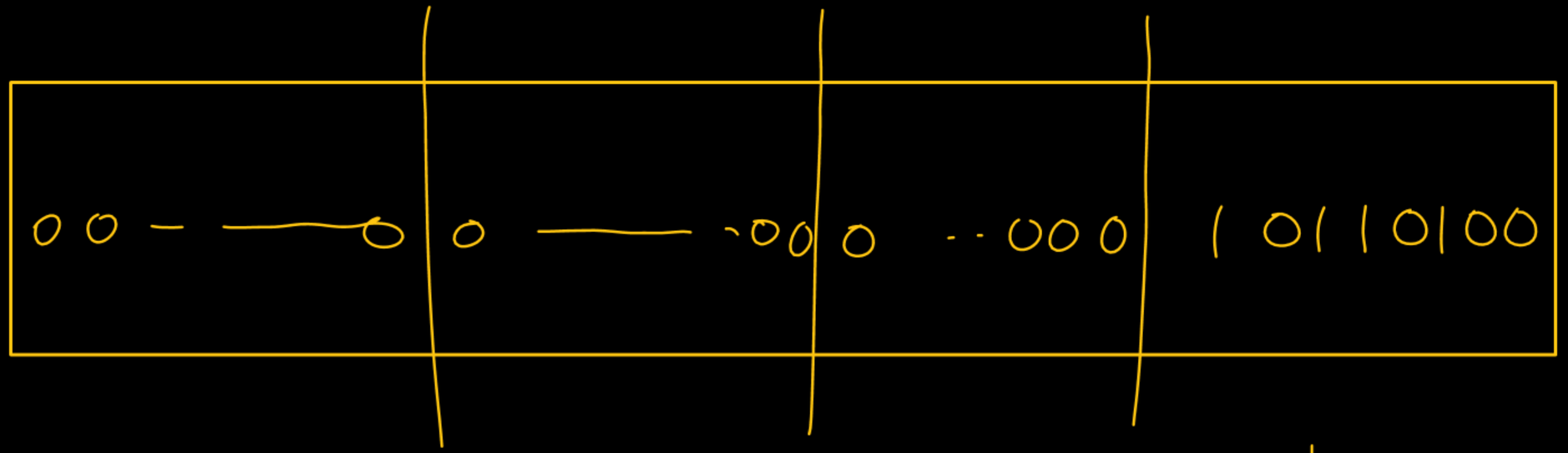
Little Endian format

O/P :


$180 = 128 + 52$

$128 + 32 + 16 + 4$



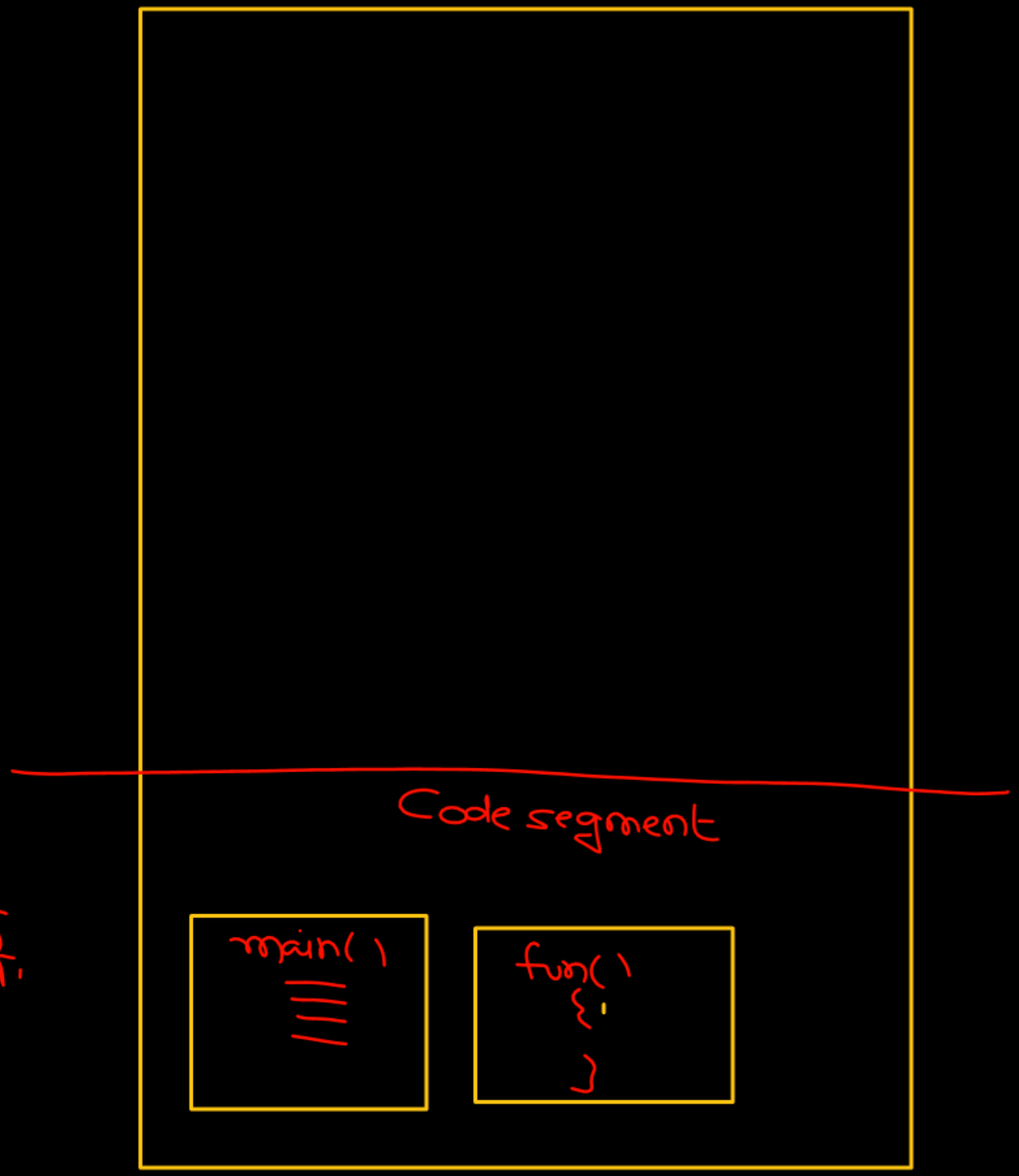

$$10110100$$
$$2^7\ 2^6\ 2^5\ 2^4\ 2^3\ 2^2\ 2^1\ 2^0$$

$$= -2^6 - 2^5 - 2^1 - 2^0 - 1$$
$$= -64 - 8 - 2 - 1 - 1$$
$$= -76$$

```
void fun(){
    ||||
    ||||
}

void main(){
    fun();
    fun();
    ||||
    ||||
    fun();
}
```

PC
instr.



# Pointer to Function (Function Pointer)

We can pass a function as an argument to other function

# Call by refrence

```
void swap(          );

void main(){
    int a=10,b=20;
    printf("a=%d,b=%d",a,b);
    swap(&a,&b);
    printf("a=%d,b=%d",a,b);
    }
```


```
void swap(int *x, int *y)
{
    int temp;
    temp = *x;
    *x = *y
    *y = temp;
}
```

# Function Pointer

① `int Add(int, int);`

② `int fun(int *);`

`int (*P)(int, int);`

`int (*P)(int *);`

How to declare pointer to function

# Reading complex Declarations

int *p ⇒ (i) int is a star p ✗

(ii) star is a int p ✗

(iii) p is a pointer to integer ✓

(1) ( ) parenthesis

(2) [ ] square brackets

(3) *

(4) Identifier — Name of var, function, array, ...

(5) data type

Priority

function, Array — 1 — L to R

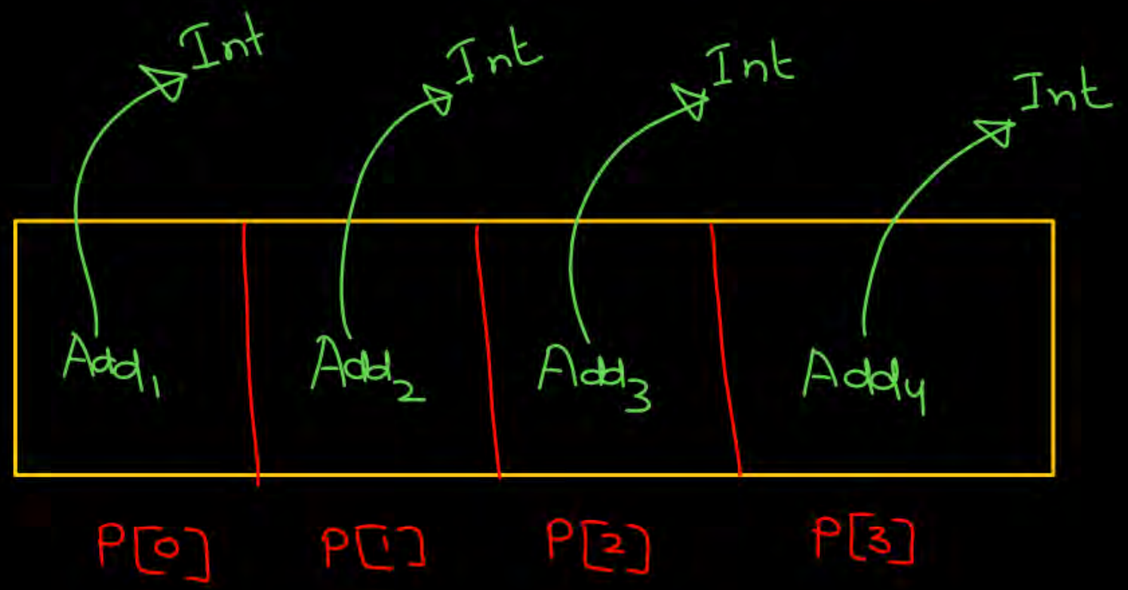
Pointer — 2 — R to L

3

1] int *(P[4])

③ ② ①

P is an array of 4 pointer to integer




P[0]
P[1]
P[2]
P[3]

} Pointer to integer

2

```
int (*P)[3];
```

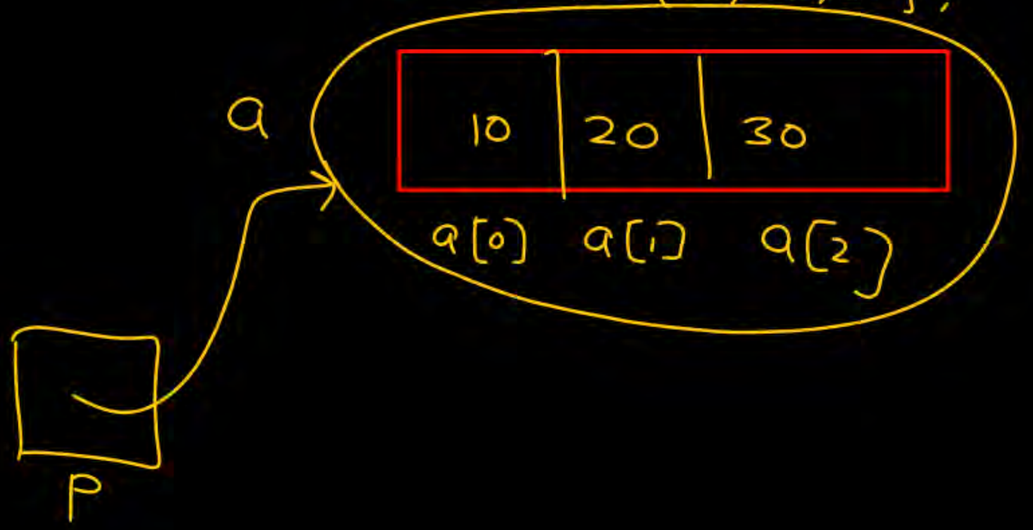
③ ① ②

P is a pointer to array of 3 integer

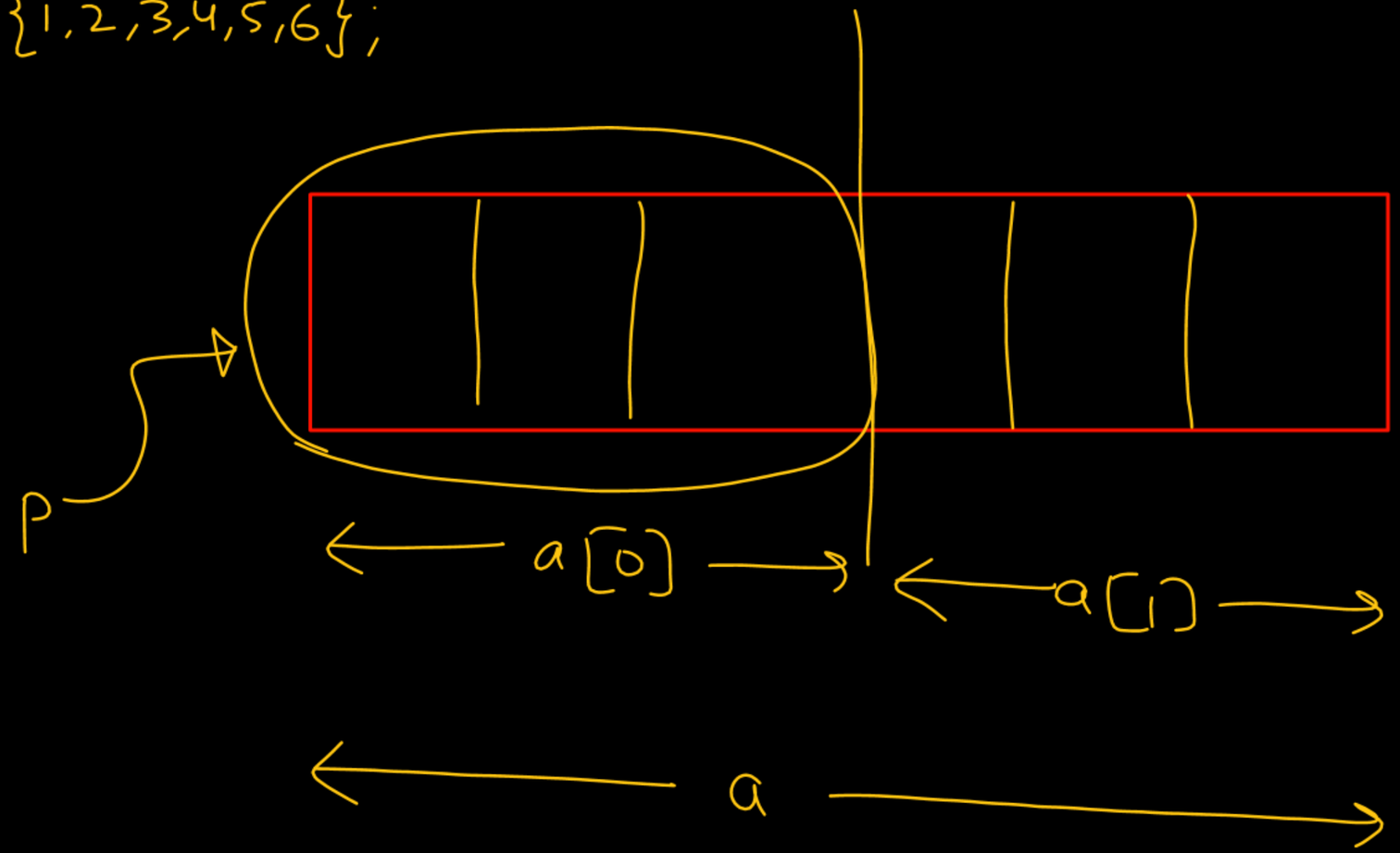
P can store address of an array of 3 integers.

```
P = &a;
```

```
int a[3] = {10,20,30};
```



```
int a[2][3] = {1,2,3,4,5,6};
int (*p)[3];
p = &a[0];
```



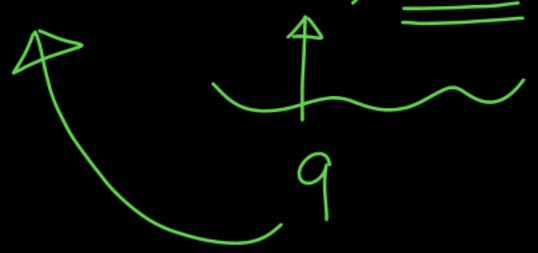
3. `int (*P)(int, int);`

P is a pointer to function → that takes 2 integer arguments and → return an integer.

`int Add(int, int);`

# Array & pointers

8




2BHK

3BHK

5 BHK


THANK YOU GW
SOLDIERS !